



Programul de studii: **CALCULATOARE**

# Aplicatie de testare online

Adaugare/editare teste online (orice domeniu)  
tip grila

Autor: Vancea Paul Alexandru

**2025**



## Cuprins

|                                             |          |
|---------------------------------------------|----------|
| <b>1. Introducere</b>                       | <b>2</b> |
| 1.1. Context general                        | 2        |
| 1.2. Obiective                              | 2        |
| 1.3. Lista MoSCoW                           | 3        |
| 1.4. Cazuri de utilizare                    | 4        |
| <b>2. Arhitectura</b>                       | <b>5</b> |
| 2.1. Front-End React                        | 5        |
| 2.2. Back-End Spring Boot                   | 5        |
| 2.3. Comunicare între front-end și back-end | 5        |
| 2.4. Baza de date MySQL                     | 6        |
| 2.5. Beneficii ale acestei arhitecturii     | 6        |

# 1 Introducere

## 1.1 Context general

Proiectul își propune dezvoltarea unei aplicații web interactive, multi-utilizator, care să realizeze sesiuni de întrebări și răspunsuri de tip quizz, cu monitorizare a rezultatelor în timp real. Nevoia de a oferi un mediu dinamic pentru evaluarea cunoștințelor și pentru colectarea feedback-ului instantaneu în timpul unei sesiuni de predare sau eveniment este scopul principal. Aplicația este concepută pentru a fi utilizată atât de un moderator/profesor cât și de un public larg (studenți/participanți).

Scopul principal este acela de a ușura procesul de interacțiune și evaluare rapidă a cunoștințelor, într-un mod accesibil și modern.

## 1.2 Obiective

Scopul acestei aplicații web este de a facilita interacțiunea didactică și evaluarea formativă. Prin dezvoltarea acestei aplicații se dorește la atingerea următoarelor obiective:

- Gestiunea completă a întrebărilor și sesiunilor de quizz de către utilizatorul cu rol de profesor.
- Efectuarea rapidă a quizz-urilor/răspunsurilor de către studenți.
- Afișarea în timp real a distribuției răspunsurilor (statistici) pe ecranul Profesorului, utilizând tehnologia WebSockets.
- Asigurarea securității datelor și a accesului bazat pe roluri (Profesor/Student) folosind JWT.
- Stocarea persistentă a întrebărilor, răspunsurilor și rezultatelor în MySQL.

### 1.3 Lista MoSCoW

În cadrul procesului de dezvoltare, stabilirea priorităților cerințelor este crucială. Metoda MoSCoW clasifică cerințele aplicației în patru categorii:

Table 1: Tabel 1.1 Lista MoSCoW

| Must Have                                                      | Should Have                                             |
|----------------------------------------------------------------|---------------------------------------------------------|
| Login/Register & Roluri (Profesor/Student)                     | Afișarea scorului final individual (Student)            |
| Operații CRUD pentru întrebare (Profesor)                      | Vizualizarea istoricului sesiunilor de quizz (Profesor) |
| Funcționalitatea de răspuns la întrebare (Student)             |                                                         |
| Afișarea <b>live</b> a distribuției răspunsurilor (WebSockets) |                                                         |

| Could Have                                         | Won't Have                        |
|----------------------------------------------------|-----------------------------------|
| Timer de expirare a sesiunii de quizz              | Întrebări asociate fiecărui admin |
| Opțiuni de personalizare a temei (dark/light mode) | Integrare cu platforme de e-mail  |
| Export de rapoarte (ex: CSV)                       |                                   |

### 1.4 Cazuri de utilizare

Această aplicație este destinată atât Profesorilor cât și Studenților. Cazurile de utilizare ilustrează modul în care cele două tipuri de utilizatori interacționează cu aplicația.

#### ADMIN (Profesor)

- Logare/Delogare
- Gestiune întrebări (CRUD: Adaugă, Modifică, Șterge)
- Gestiune sesiuni de Quizz (Creare sesiune, Activare, Închidere)
- Vizualizare rezultate **LIVE** (Distribuția răspunsurilor)
- Vizualizare scoruri finale (după încheierea sesiunii)

#### GUEST (Student)

- Logare/Delogare
- Efectuare Quizz Online (Răspuns la întrebări)

- Vizualizare scor personal (după corectare/finalizarea sesiunii)

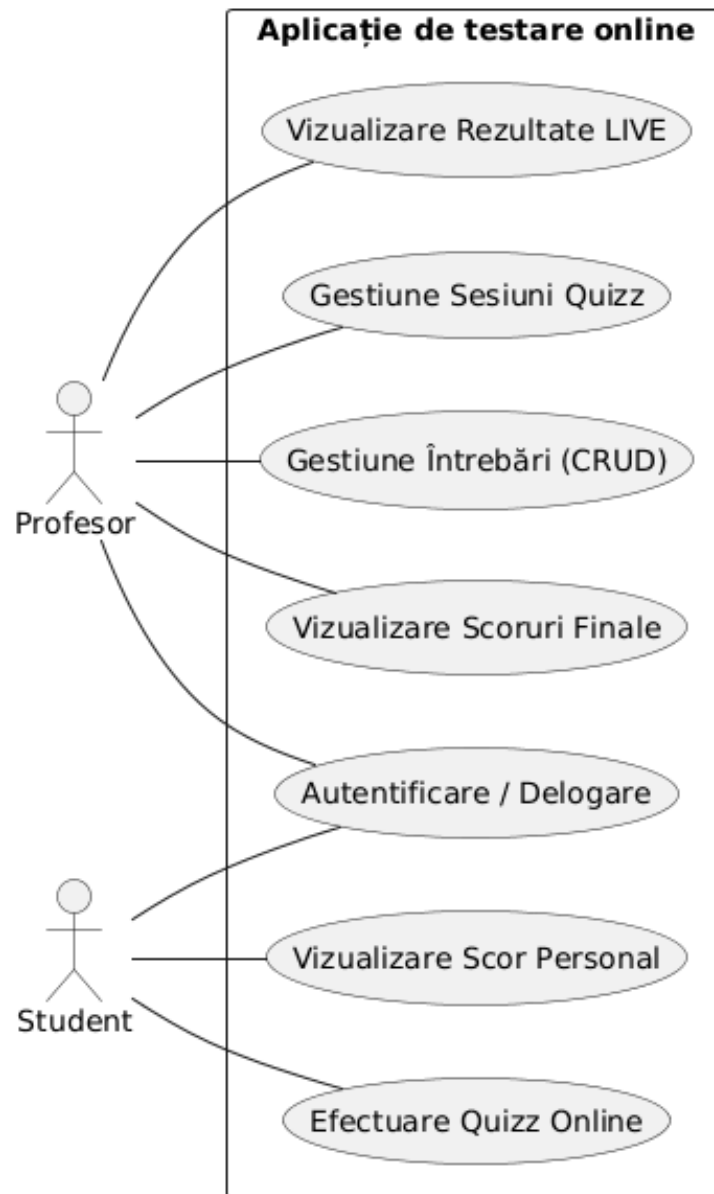


Figure 1: Figura 1.1 Diagrama cazurilor de utilizare

## 2 Arhitectura

Arhitectura aplicației este concepută pentru a asigura o performanță de înaltă calitate, scalabilitate și o interfață utilizator prietenoasă. Cu un front-end dezvoltat în React și un backend bazat pe Spring Boot, această arhitectură modernă îmbină tehnologii de ultimă generație pentru a oferi o experiență coerentă și robustă utilizatorilor.

### Arhitectura Aplicației "Aplicație de testare online"

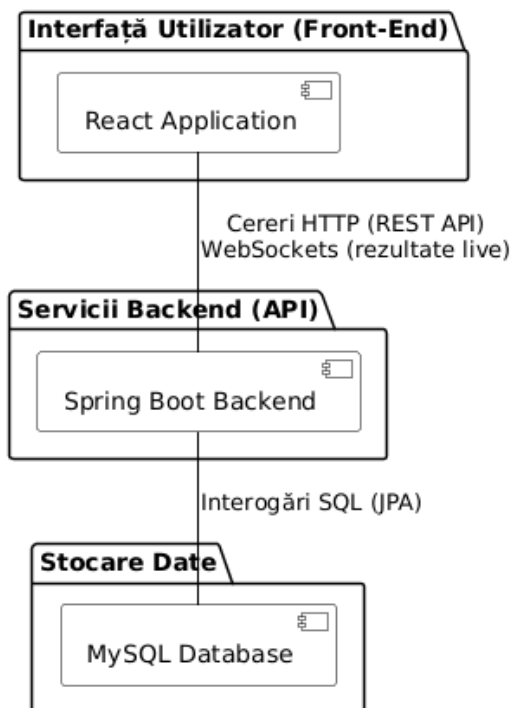


Figure 2: Figura 2.1 Arhitectura aplicației

### 2.1 Front-End React

Partea de frontend este dezvoltată în React, pentru a beneficia de modularitate, flexibilitate și viteză de dezvoltare. Componentele React vor gestiona starea aplicației și vor facilita interacțiunea fluidă a utilizatorului.

## 2.2 Back-End Spring Boot

Spring Boot reprezintă coloana vertebrală a back-end-ului, oferind un cadru robust pentru dezvoltarea serviciilor RESTful. Acesta va gestiona logica de business, securitatea, și comunicarea cu baza de date prin Spring JPA.

## 2.3 Comunicare între Front-End și Back-End

Comunicarea clasică (CRUD) se va realiza prin intermediul cererilor HTTP, cel mai probabil utilizând o bibliotecă precum Axios. Comunicarea în timp real (pentru actualizarea rezultatelor live) se va realiza prin **WebSockets** (Spring WebSockets / STOMP).

## 2.4 Baza de Date MySQL

MySQL este ales ca sistem de gestionare a bazelor de date datorită popularității sale, ușurinței în utilizare și performanței dovedite în aplicațiile web. Interfața Spring Data JPA simplifică interacțiunea cu baza de date.

## 2.5 Beneficii ale acestei arhitecturii

- **Timp Real:** Integrarea WebSockets permite actualizarea instantanee a rezultatelor Profesorului, îmbunătățind interactivitatea.
- **Securitate:** Utilizarea Spring Security și JWT asigură o metodă robustă și *stateless* de gestionare a autentificării și autorizării.
- **Eficiență în dezvoltare:** Utilizarea framework-urilor populare (React și Spring Boot) contribuie la o dezvoltare rapidă și ușoară.
- **Scalabilitate:** Arhitectura permite extinderea facilă a capacităților aplicației pe măsură ce numărul de utilizatori crește.